Flight controller ⇄ (Raspberry Pi companion) ⇄ MAVProxy ⇄ TCP/UDP ⇄ Cloud dashboard

By Eng Achraf RAHIBE

What this project is about

You already have a cloud/edge mission-planner + scheduler dashboard (Storm-style multi-drone control). Today, a real drone would be:

Your request for me (3-day task) is to deliver the **simulation equivalent** of that real-drone setup:

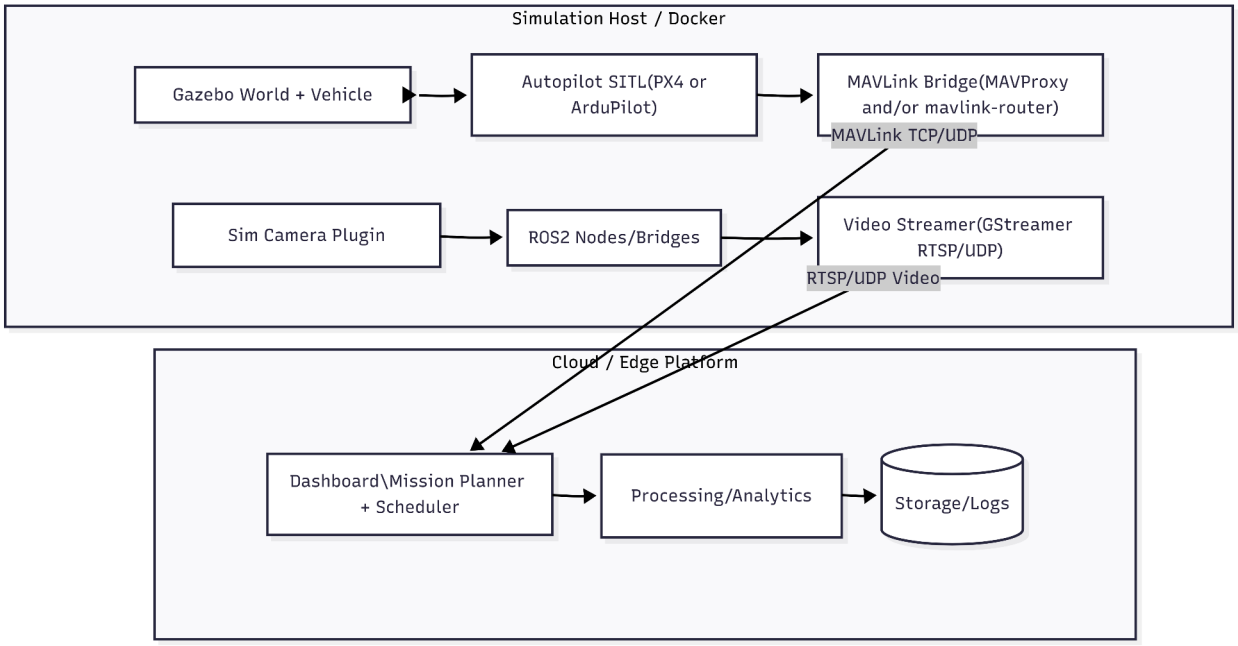
- A **Gazebo world + vehicle model** (single drone first, scalable to multiple)
- A **SITL autopilot** that behaves like a flight controller
- A **MAVLink transport bridge** that makes the simulated drone publish telemetry to your dashboard **exactly like a real FC**, via **TCP/UDP endpoints** (using MAVProxy routing).
- A **camera pipeline** in simulation that delivers video to the dashboard in a practical way (recommended: RTSP/UDP via GStreamer, not "raw video over MAVLink").

Key design choice (so it scales to many drones)

For multi-vehicle support, I'll structure the network so each drone has:

- Unique **MAVLink System ID + port mapping**
- A routing layer that can forward MAVLink to one or many consumers (dashboard, logger, debug tools), typically via **mavlink-router** (UDP/TCP).

System architecture (overview)



Notes

- MAVProxy can forward MAVLink to remote endpoints using **tcpout/udpout**, matching your "real FC + companion" routing pattern.
- For video, QGC and PX4 docs explicitly rely on **GStreamer RTSP/UDP** patterns; same approach fits a custom dashboard.

What I will deliver + Gazebo world I/O

Deliverables (in 3 days)

D1 - Simulation baseline

- Gazebo world boots reliably (map/terrain, lighting, basic environment)
- Drone model spawns and SITL connects and flies in sim (sanity: arm/takeoff/land)
- MAVLink telemetry visible locally (debug)

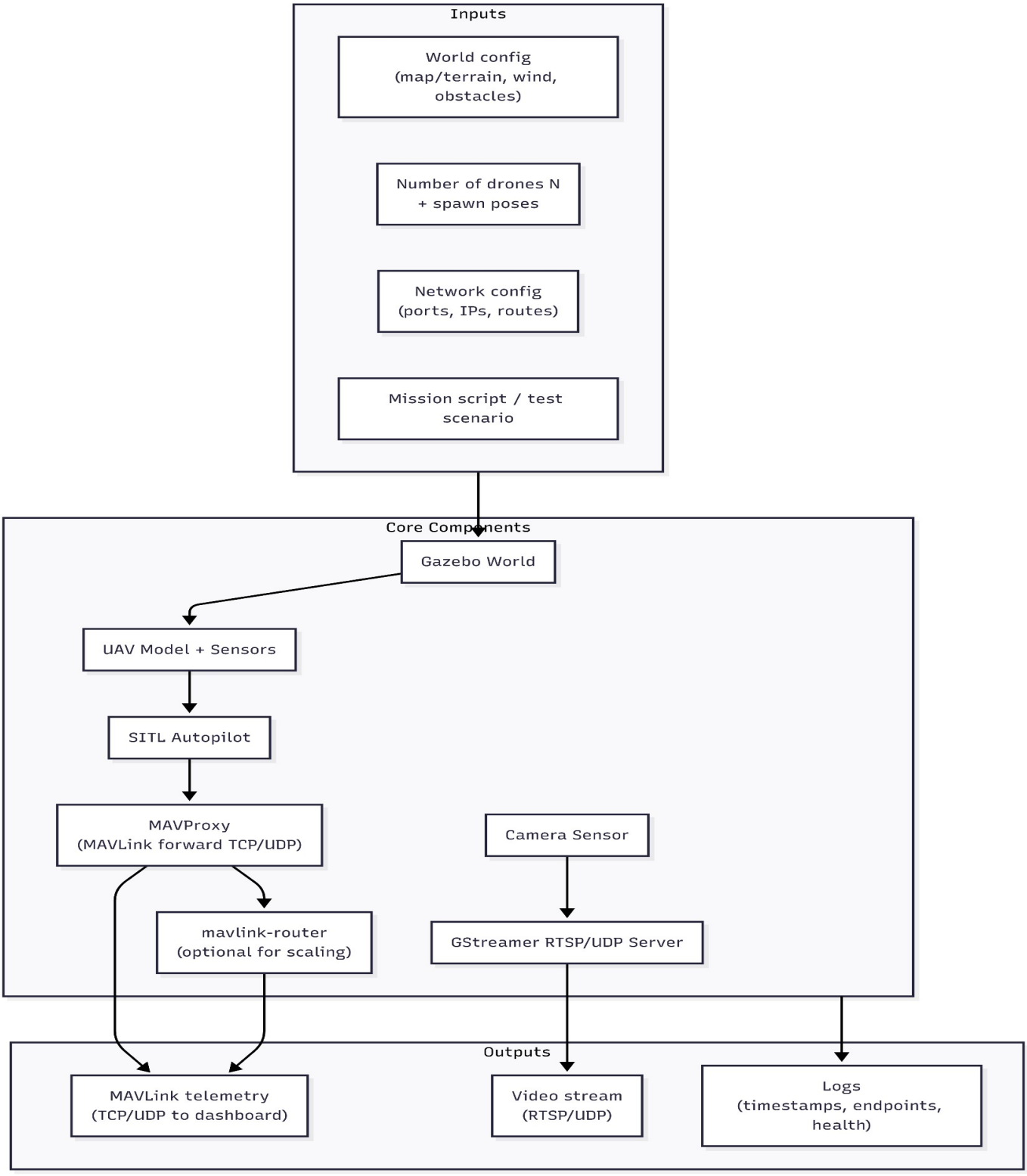
D2 - Dashboard connectivity

- MAVLink forwarded to your dashboard via **TCP/UDP** (same "companion → cloud" pattern)
- Stable endpoint config + run scripts (repeatable, Docker-friendly)
- Optional: add **mavlink-router** layer for cleaner multi-endpoint + future multi-drone routing

D3 - Camera + multi-drone-ready structure

- Sim camera enabled and streaming to dashboard (recommended: RTSP/UDP via GStreamer)
- Multi-drone template: port plan + system-id plan + "launch N drones" approach

Gazebo world: inputs/outputs + tools


Inputs
World config (map/terrain, wind, obstacles)
Number of drones N + spawn poses
Network config (ports, IPs, routes)
Mission script / test scenario
Core Components
Gazebo World
UAV Model + Sensors
SITL Autopilot
MAVProxy (MAVLink forward TCP/UDP)
mavlink-router (optional for scaling)
Camera Sensor
GStreamer RTSP/UDP Server
Outputs
MAVLink telemetry (TCP/UDP to dashboard)
Video stream (RTSP/UDP)
Logs (timestamps, endpoints, health)

Checklist subtasks (quick briefing)

- Confirm autopilot choice: **ArduPilot SITL + Gazebo** (fits MAVProxy naturally) or PX4 SITL (still MAVLink, different tooling).
- Define dashboard MAVLink interface: which messages you consume + expected rate
- Lock MAVLink transport: UDP vs TCP, ports, IP (cloud ingress) - implement via MAVProxy forwarding
- Camera streaming method: RTSP/UDP via GStreamer (recommended)
- Multi-drone plan: per-drone ports + SYSID + routing layer (mavlink-router)